

NOMAD – справочник API ядра

Системные вызовы, capability-модель, память и куча userspace, FPU-семантика, SMP/IPI, формат исполняемых файлов и crt0. Синхронизировано с abi.rs (workspace 0.1), октябрь 2026.

1. Архитектура

Ядро разделено на архитектурно-независимый слой `kernel_base` (capability-менеджер, трейты памяти/IOMMU/прерываний/IPI, `exec`-слой, TCB), платформенный `kernel_x86` (страничные таблицы, LAPIC/IPI, IRQ-диспетчер, `entry`-стабы `cswitch`), планировщик `kernel_sched`, `exec`-домен `kernel_exec` (ELF, `spawn`, `auxv`), загрузчик `kernel_limine` (bin `cintos_kernel`) и `proc`-макро `syscall_macros`. Userspace: библиотека `cintos-user` (ABI-обёртки, `crt0`, куча, L4-IPC-транспорт, C ABI) и каркас `init`-сервера `init_system`.

Дисциплина ядра – `slab-first`: все аллокации через `SlabBox` (включая `SlabBox<T>` с учётом `weak`-семантики), реестры – с `per-entry` блокировкой. Загрузка – `Limine` (BIOS+UEFI), ELF-модули `boot`-ростера получают `peer`-слоты `cspace 2+i` по порядку в `limine.conf`. Ядро SMP: AP просыпаются через SIPI, таймер AP – локальный LVT LAPIC, межъядерные события – IPI (см. §7).

2. Конвенция системного вызова (x86_64)

Элемент	Значение
Номер сисколла	RAX
Аргументы 1..6	TDI, RSI, RDX, R10, R8, R9 (порядок SysV; R10 вместо RCX)
Возврат	RAX: 0/результат – успех; старший бит (0x8000_0000_0000_0000) – ошибка
Портится	RCX, R11 (железо SYSCALL), RAX
Инструкция	SYSCALL; возврат – SYSRET (<code>entry</code> -стаб порта)
Успех с результатом	например, <code>id</code> созданной capability; <code>id 0</code> валиден – различение по старшему биту

Коды ошибок (константы в `kernel_base::traits::syscall` и `cintos_user::abi::result`): 1 `NO_CURRENT_TASK`, 2 `RIGHTS_DENIED` (отказ групповых прав неймспейса), 3 `NOT_FOUND`, 4 `SLOT_OCCUPIED`, 5 `SLOT_EMPTY`, 6 `CAP_REVOKED`, 7 `RIGHTS_EXCEEDED`, 8 `SLAB`, 9 `QUOTA`, 10 `IDS_EXHAUSTED`, 11 `INVALID_ARG`, 12 `INTERNAL`, 13 `NOT_IMPLEMENTED` (сисколл известен, но не реализован), 14 `BUSY` (ресурс занят: линия IRQ занята, `FREE_PAGES` под IOMMU-пином), 15 `TIMEOUT` (истёк дедлайн `IPC_WAIT`).

3. Таблица системных вызовов (плоская нумерация)

NR	Имя	Аргументы (по порядку)	Возврат/семантика
0	<code>SchedYield</code>	–	добровольная передача ядра
1	<code>SchedRegisterTask</code>	<code>entry</code> , <code>code_size</code> , <code>stack_size</code> , <code>task_cap</code>	регистрация <code>runtime</code> -метаданных; ок – задача готова к запуску
2	<code>SchedDestroyTask</code>	<code>task_cap</code>	уничтожение задачи; <code>self-exit</code> – задача передаёт собственный <code>id</code> (<code>crt0::exit</code>)
3	<code>SchedBlockOnObject</code>	<code>object_id</code>	сон на объекте ожидания (<code>OneShot</code>)
4	<code>SchedReleaseObject</code>	<code>object_id</code>	пробуждение ожидающих объект
5	<code>AllocPages</code>	<code>pages</code>	выделить страницы СВОЕЙ задаче; ядро само мапит их в AS; возврат – VA; <code>MEMORY_ALLOC</code> + квота группы; окна между вызовами НЕ смежны

NR	Имя	Аргументы (по порядку)	Возврат/семантика
6	FreePages	vaddr	снять аллокацию по базовому VA; E_BUSY, если регион записан IOMMU (DMA-buf)
7	MountCapRegion	регион по capability	смонтировать MMIO- или shm-регион в AS; возврат – VA; MMIO_MAP
8	UnmountCapRegion	vaddr	снять смонтированный регион
10	IpcSend	слот получателя, msg, размер, caps-массив, число caps	синхронный rendezvous; пересылка capability – map item'ами (внутри сообщения)
11	IpcWait	слот отправителя ANY, буфер, ёмкость, база приёмного окна, размер окна	open/closed wait; приём map item'ов в окно; дедлайн – иначе E_TIMEOUT
16	CapCreateNamespace	dst_slot, max_tasks, max_mem, badge, rights	создание неймспейса (группы); возвращает id capability; требует CAP_MANAGE
17	CapCreateIpcPool	owner_task, dst_slot	capability на пул IPC-памяти; CAP_MANAGE MEMORY_ALLOC
18	CapCreateMmio	owner_task, dst_slot, phys, pages	capability на MMIO-регион (phys_guard: acpi-allow-list для ACPI-диапазонов); CAP_MANAGE MMIO_MAP
19	CapCreateIrq	owner_task, dst_slot, line, trigger	IRQ v2: капа на ЛОГИЧЕСКУЮ линию (GSI/MSI), trigger: 0=edge, 1=level; линия обязана быть свободна (иначе E_BUSY); успех – занята владельцем, замаскирована
20	CapMint	src_task, src_slot, dst_task, dst_slot, rights	производная копия (права $\subseteq$ источника); требует CAP_MINT + право Mint у капы
21	CapClone	src_task, src_slot, dst_task, dst_slot	копия в той же мембране; требует право Clone
22	CapRevoke	task, slot	ревок мембраны слота – протухают все производные
23	CapDestroy	task, slot	ревок + снятие записи
24	– (удалён)	–	бывший CapTransfer удалён (ambient authority); пересылка capability живёт в IPC map item'ах; НОМЕР НЕ ПЕРЕИСПОЛЬЗОВАТЬ
25	CapCreateShared	owner_task, dst_slot	капа на разделяемый регион СОБСТВЕННОЙ памяти (shm); физику резолвит ядро; получатель монтирует через NR 7
26	CapCreateFaultEndpoint	dst_slot	фолт-эндпоинт: фиксирует ТЕКУЩУЮ задачу как обработчика фолтов; CAP_MANAGE FAULT_HANDLE
27	FaultSetEndpoint	ep_slot, target_slot	привязка обработчика к задаче-цели (её TaskTCB-слот, право Send); TASK_CREATE FAULT_HANDLE; повтор – замена
28	IrqWait	caps_ptr, caps_len (1..=8), mask_ptr	IRQ v2: сон до срабатывания линий; линии адресуются капами IrqLine (слоты cspace); ядро пишет [count][line0..] в mask_ptr; WAIT размаскирует набор (claim маскирует)
29	TaskStats	буфер	снэпшот счётчиков задачи + глобальные тики/частота; самоинспекция без прав (STATS_READ для чужих)

NR	Имя	Аргументы (по порядку)	Возврат/семантика
30	FaultReply	target_task_cap, new_rip, new_rsp	resume упавшей: 0/0 – повторить упавшую инструкцию; иначе продолжить с нового адреса/стека; FAULT_HANDLE; только зарегистрированному обработчику после приёма сообщения
32	IommuCreateDomain	dst_slot	DMA-домен; DMA_ATTACH; возвращает id capability
33	IommuAttachDevice	task, slot, bus, dev, func	присоединение PCIe-устройства (BDF от userspace)
34	IommuMapDma	task, slot, iova, phys, pages, prot	DMA-маппинг (second-stage)
35	IommuUnmapDma	task, slot, iova, pages	снятие DMA-маппинга (снимает пин DMA-buf)
36	IommuCreatePasidSpace	domain_task, domain_slot, mode, dst_slot	PASID-пространство; mode: 0=Dedicated, 1=OwnAddressSpace (SVA)
37	IommuAllocPasid	space_task, slot, ceiling, dst_slot	выделить PASID-капабилити с потолком одновременных пользователей (сверх – E_QUOTA)
38	IommuFreePasid	pasid_task, slot	уничтожить PASID, сняв все привязки устройств
39	IommuBindPasidDevice	pasid_task/slot, dev BDF	устройство-пользователь в PASID (квота – E_QUOTA)
40	IommuUnbindPasidDevice	pasid_task/slot, dev BDF	отвязать устройство от PASID
41	IommuMapVa	pasid_task, slot, gva, phys, pages, prot	first-stage маппинг через PASID-капабилити (dedicated-режим)
42	IommuUnmapVa	pasid_task, slot, gva, pages	снятие first-stage маппинга
43	IommuDestroyPasidSpace	space_task, slot	уничтожение пространства (только без живых PASID)
44	IommuDestroyDomain	domain_task, slot	уничтожение DMA-домена
45	IommuDetachDevice	domain_task, slot, BDF	отвязка устройства от домена
46	DbgLogRead	буфер	чтение дельты лога ядра (консоль init-сервера)
47	DbgLogWrite	ptr, len	запись строки задачи в лог ядра (serial + кольцевой буфер); база log!/logln!
48	TaskCreate	ns_cap, image_slot, dst_slot, ...	спавн ELF-образа boot-модуля в неймспейс; authority: капа неймспейса + капа TaskImage; возврат – task_cap_id ребёнка (и в dst_slot создателя)
49	TaskCreateFromMem	ns_cap, ptr, size, dst_slot, ...	ехес ELF из ЧИТАЕМОЙ памяти вызывающего (бинарь от файлового сервера map item'ом или ALLOC_PAGES-буфер); ядро снимает снапшот ДО разбора; TASK_CREATE у ОБОИХ неймспейсов
50	IommuMapDmaVa	домен task/slot, iova, va, pages, prot	DMA-buf: маппинг ИЗ памяти вызывающего; ядро резолвит VA-физику по VmapRegion и пинит регион (FREE_PAGES пина – E_BUSY); va обязан лежать в одной ALLOC_PAGES-аллокации
51	IrqMsiAlloc	count, first_dst_slot, msgs_ptr	выделить count MSI-линий (message-backed); корневые капы в слоты; MSI-сообщения [line, address, data, trigger] в буфер; линия замаскирована до первого WAIT

NR	Имя	Аргументы (по порядку)	Возврат/семантика
52	IrqRelease	slot	владелец возвращает линию платформе (маска + снятие записи + тумбстоун капы)

4. Модель прав

Высокогранулярные права capability (DirectCapabilityRights, биты маски в CapMint): Clone=1, Mint=2, Send=4. Send – передача через IPC map item'ы. Групповые права неймспейса (NamespaceRights, битмаска rights в CapCreateNamespace, u64) – потолок для всех потоков группы: TASK_CREATE=1, MEMORY_ALLOC=2, MMIO_MAP=4, IRQ_BIND=8, IPC_SEND=16, CAP_TRANSFER=32, CAP_MINT=64, CAP_MANAGE=128, DMA_ATTACH=256, STATS_READ=512, FAULT_HANDLE=1024. Неизвестные биты ядро отбрасывает, права ребёнка всегда $\subseteq$ прав создателя.

Отказ неймспейса имеет приоритет: даже корректная capability не даст доступ, если группового права нет. Разрешение объекта – потолок класса: MemoryIPCPool→MEMORY_ALLOC, MMIO→MMIO_MAP, IrqLine→IRQ_BIND, TaskTCB/TaskImage→TASK_CREATE, Namespace→CAP_MANAGE, Iommu*/Pasid*→DMA_ATTACH, FaultEndpoint→FAULT_HANDLE. Юзерспейс-хелперы:

cintos_user::cap – типизированные обёртки (create_namespace/ipc_pool/mmio/irq/shared/fault_endpoint, mint/clone/revoke/destroy, mount_region/unmount_region, типы Rights/NamespaceRights/IrqTrigger) и cintos_user::mem (alloc_pages/free_pages); C-эквиваленты – nomad_cap_* в nomad.h. Перепутать сырые u64 больше нельзя: модуль cintos_user::handle вводит различные типы Slot (слот cspace), CapId (id капы ядра), TaskCap (TaskTCB-капы), Va/Phys (адреса), Pages (страницы) – смешение слота с капой или VA с числом страниц ловится компилятором; на границах (wire/auxv/C-ABI) – явные new()/raw().

5. Память и куча userspace

Ядро НЕ экспортирует mmap: задача запрашивает AllocPages(pages) (NR 5) и получает VA – отображение ядро выполняет само; окна между вызовами не обязаны быть смежными. FreePages (NR 6) снимает аллокацию по базовому VA; регион, запиненный под DMA-buf (NR 50), освобождается ошибкой E_BUSY до IommuUnmapDma. Монтирование чужих регионов (MMIO, shm) – только по capability: MountCapRegion/UnmountCapRegion (NR 7/8). Квоты группы ограничивают суммарную память; исчерпание – E_QUOTA.

Куча userspace (cintos_user::heap): #[global_allocator] KernelHeap поверх ALLOC_PAGES – арена стартует ПУСТОЙ (нулевой резерв), первый аллок растит её вызовом AllocPages(max(4 страницы, потребность)) и добавляет чанк в address-ordered free-list со слиянием соседей. Размер страницы – из auxv AT_PAGESZ. Обратного пути нет: FREE_PAGES у кучей не вызывается, вся память возвращается ядру при SCHED_DESTROY_TASK. Диагностика – KernelHeap::current_stats() (chunks/used/free). В prelude экспортированы Vox/String/Vec/vec!/format! из alloc – бинам достаточно use cintos_user::prelude::*.

6. FPU/SSE: eager FXSAVE/FXRSTOR

Ядро включает CR4.OSFXSR|OSXMMEXCPT и сохраняет FPU-состояние eager – без lazy-фолта #NM: fxsave64 при каждом входе из ring3 (syscall после записи кадра, IRQ/фолт – только для ring3-входов), fxrstor64 при каждом возврате в ring3. Скретч – per-CPU FpuScratch 512 Б, выравнивание 16 (указатель в PerCpuFixed, gs:[0x28]); постоянное хранилище – FpuArea 512 Б в TCB (перекачка per movsq при переключении задач). x86_64 baseline гарантирует SSE2, поэтому CPUID-gate не нужен.

Контракт userspace: регистры x87/MMX/XMM переживают сисколлы, прерывания, фолты и переключения задач – core::fmt и любой SSE-код можно применять вокруг syscall'ов. Новая задача стартует детерминированно: fninit + ldmxcsr 0x1f80 (шаблон TCB: FCW=0x037f – все x87-исключения замаскированы, MXCSR=0x1f80 – все SSE-исключения замаскированы).

7. SMP и межъядерные прерывания

IPI-контроллер (kernel_x86::ipi, X86IpiController в ArchImplementation) реализует переносимый контракт kernel_base::traits::ipi поверх LAPIC ICR (xAPIC MMIO / x2APIC MSR). Службные векторы IDT (не пересекаются с линиями 32+GSI и MSI-пулом): 250 – локальный LVT-таймер LAPIC (тик AP), 252 – TLB-shootdown, 253 – Halt (зарезервирован), 254 – Reschedule (кик цикла планировщика).

Протокол TLB-shootdown – синхронный, без аллокаций: отправитель после очистки PTE и локального invlpg кладёт запись {root, virt, pages} в очередь фиксированной ёмкости, увеличивает поколение и шлёт IPI 252 онлайнным ядрам; получатель вычищает invlpg только записи с корнем, совпадающим с ТЕКУЩИМ CR3 (GLOBAL не ставится, смена CR3 чистит TLB целиком), и подтверждает Release-записью наблюдаемого поколения. Диапазоны сверх капы INVLPg и переполнение очереди вырождаются в полную вычистку (перезапись CR3). Отправитель ждёт asked-поколений (bounded-спин с re-send); по таймауту – громкий лог и выход, не hang.

8. Формат исполняемых файлов и crt0

Образы – ELF64 с EI_OSABI = 0xC1 (NOMAD), ET_EXEC, e_machine = EM_X86_64 (62); Linux-ELF отклоняются гейтом OS ABI (патч ставит scripts/patch_osabi.py). Формат подключаемый: трейт ExecFormat (probe/parse), нейтральное ImageInfo (entry + сегменты) – загрузчику arch-слоя всё равно, что внутри. Раскладка стартового стека (начальный RSP указывает на argc, кратен 16):

```
stack_top (высокие адреса)
  строки argv/envp (NUL-терминированные)
  auxv: (tag,val)*16B ... AT_NULL
  envp: указатели, NULL
  argv: указатели, NULL
  argc: u64                <- начальный RSP (кратен 16)
```

AUXV: 7=PAGESZ, 9=ENTRY, 0xC170_0001=SELF_CAP, 0xC170_0002=NS_CAP, 0xC170_0003..0007=FB_ADDR/PITCH/WIDTH/HEIGHT/BPP (кадровый буфер), 0xC170_0008=ACPI_RSDP (физадрес; ACPI-диапазоны монтируются CAP_CREATE_MMIO из acpi-allow-list ядра).

Bootstrap-слоты cspace: 0 = self-TCB (полные права), 1 = корневой неймспейс, 2+i = peer-TCB boot-ростера (у динамической задачи слот 2 – родитель), 32+j = капы TaskImage boot-образов у init.

crt0 несёт lang-items #[lang="start"] и #[lang="termination"]: бин – обычный #[no_std] fn main(), без no_main-бойлерплейта. Возврат из main (Termination для ()/i32/u32) = self-exit через SCHED_DESTROY_TASK со своим капом; паник-хендлер – аварийный self-exit. Аксессуары: args(), argv_at(i), envp(), auxv_get(tag), bootstrap() → {self_cap, namespace_cap, page_size, entry}, acpi_rsdp_phys(), fb_aux(), stack_base(), exit(code). C-совместимость: staticlib libcintos_user.a + include/nomad.h (C-символы nomad_*; C-main = main(long argc, char** argv)).

9. Доступ к ресурсам из userspace

Операция	Как
Узнать себя	crt0::bootstrap() -> { self_cap, namespace_cap, page_size, entry }
Выход	вернуть код из main (crt0 делает self-exit) или crt0::exit(N)
Лог в serial ядра	log! / logln! из prelude (DBG_LOG_WRITE)
Куча (Vec/String/Box)	use cintos_user::prelude::* – рост через ALLOC_PAGES, арена пустая до первого аллока
Сырая память	mem::alloc_pages(Pages) -> Va; mem::free_pages(Va) (NR 5/6)
MMIO	cap::create_mmio(TaskCap, Phys, Pages, Slot) -> cap::mount_region(Slot) -> Va (NR 18/7); снять – cap::unmount_region(Va)

Операция	Как
Производные капы	cap::mint/clone (обе стороны – TaskCap в cspace вызывающего), revoke/destroy(TaskCap, Slot); права – типы cap::Rights / cap::NamespaceRights
Shared memory	mem::alloc_pages(Pages) → cap::create_shared(Va, Pages, Slot) → капа через IPC map items → получатель cap::mount_region(Slot из приёмного окна)
Создать задачу	TaskCreate (образ boot-модуля) или TaskCreateFromMem (ELF из своей памяти)
Драйвер устройства	CapCreateMmio (ядро) + IOMMU-инвокации 32..45; DMA из своей памяти – IommuMapDmaVa (50)
SVA (свои указатели в DMA)	IommuCreatePasidSpace mode=1 + IommuAllocPasid + BindPasidDevice + MapVa
Ждать IRQ	CapCreateIrq (линия) + IrqWait([слоты], буфер): после пробуждения [count][line...]; MSI – IrqMsiAlloc
Обработчик фолтов	CapCreateFaultEndpoint + FaultSetEndpoint; фолты #DE/#BP/#OF/#UD/#GP/#PF/#MF/#AC/#XF приходят через IPC; ответ – FaultReply (0/0 = повтор инструкции)
Статистика	TaskStats – счётчики задачи + тики/частота ядра
ACPI	crt0::acpi_rsdprphys() + CapCreateMmio по allow-list (XSDT, MCFG, DRHD)

10. Проволочный формат IPC-сообщений (без сериализационных фреймворков)

Ядро – чистый L4-транспорт: payload сисколлов IPC_SEND/IPC_WAIT НЕ разбирается, формат тела – соглашение юзерспейса. Решение (после опытной эксплуатации mini-FlatBuffers-рантайма): таблицы и сериализационные фреймворки сняты, тело – фиксированный заголовок + payload (seL4-стиль). Аргументы: no_std/no-alloc юзерспейс (официальный flatbuffers-крейт тянет std/alloc – google/flatbuffers#7089 не решён); буфер приёма приходит от враждебной задачи – проверка границ тривиальна и тотальна против структурного обхода; C-совместимость без зеркала библиотеки; потолок payload 512 байт (большое – через shm-канал мимо ядра), поэтому выразительность таблиц не окупает их стоимость.

Формат тела (little-endian): [label u64][payload_len u64][payload]; label – тег типа сообщения (MR0 у Лидтке), payload – непрозрачные байты. Зеркала констант: cintos_user::ipc (BODY_HDR = 16, MAX_MSG = 512) – kernel_base::ipc::endpoint::MAX_MSG – nomad.h NOMAD_MSG_MAX. Ядро кодирует фолт-сообщения тем же форматом (FaultInfo::encode: label = FAULT_LABEL, payload = 5×u64, 56 байт) – обработчик разбирает фолт обычным ipc::wait/parse_received. Доставка в буфере приёма: [sender u64][body_len u64][caps_count u64][слоты×N][тело].

Верификация: parse_received проверяет только границы (заголовок доставки и тела, payload_len в пределах тела) – любое усечение/порча даёт None без паник (усечение каждого байта покрыто тестами); заимствование payload – zero-copy из буфера приёма. Опциональность и эволюция схем – уровень ПРОТОКОЛА, не транспорта: новые типы сообщений = новые метки label; изменения формата payload – версия в label или резервные поля. Если протоколы дорастут до вариативных структур – отдельный шаг: .fbs-схемы + кодген (flatc/planus) на host-стороне, проволочный транспорт при этом не меняется.

11. Ограничения текущей версии

SchedRegisterTask (NR 1) – kernel-internal регистрация runtime; nr 24 удалён навсегда (ambient authority). Таймауты: дедлайн есть у IPC_WAIT (E_TIMEOUT); SchedBlockOnObject и IrqWait ждут неограниченно. PASID ограничен 512 на юнит; Intel QI и AMD GN-инвалидации требуют верификации на QEMU. Фолт-эндпоинты доставляют фолты ring3 обработчику через IPC-транспорт; маппинг страниц ИЗ обработчика (пейджер) – дорожная карта. init_system –

каркас: веха 0.1-beta забирает у kernel_limine регистрацию серверов (spawn_boot_servers)
– ядро спавнит только init, остальное init раскатывает сам по росту boot-модулей.
Дорожная карта: расширение ExecFormat, Endpoint-capability для IPC.